

Password Reset Token Lifetime

- **DataProtectorTokenProvider** class generates both Password Reset and Email Confirmation tokens
- Token life span is controlled by **DataProtectionTokenProviderOptions** class
- The default token life span is **1 day**

Both, password reset token and email confirmation token are generated by the built-in `DataProtectorTokenProvider` class. In our [previous video](#), we discussed in detail, how this class generates and validates these tokens.

So, it is the `DataProtectorTokenProvider` class that generates the password reset token. The token life span is controlled by `DataProtectionTokenProviderOptions` class. You can see this yourself, if you look at the source code of `DataProtectorTokenProvider` class at the following link.

<https://github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProvider.cs>



The screenshot shows the source code of the `DataProtectorTokenProvider` class. Annotations highlight the `options` parameter in the constructor and the `Options` property, both of which are of type `DataProtectionTokenProviderOptions`. A note explains that this class controls the lifespan of the tokens generated.

```
public class DataProtectorTokenProvider<TUser> : IUserTwoFactorTokenProvider<TUser> where TUser
{
    /// <summary>
    /// Initializes a new instance of the <see cref="DataProtectorTokenProvider{TUser}"> class.
    /// </summary>
    /// <param name="dataProtectionProvider">The system data protection provider.</param>
    /// <param name="options">The configured <see cref="DataProtectionTokenProviderOptions"/>.</param>
    public DataProtectorTokenProvider(IDataProtectionProvider dataProtectionProvider, IOption<DataProtectionTokenProviderOptions> options)
    {
        constructor
        {
            if (dataProtectionProvider == null)
            {
                throw new ArgumentNullException(nameof(dataProtectionProvider));
            }
            Options = options?.Value ?? new DataProtectionTokenProviderOptions();
            // Use the Name as the purpose which should usually be distinct from others
            Protector = dataProtectionProvider.CreateProtector(Name ?? "DataProtectorTokenProvider");
        }

        /// <summary>
        /// Gets the <see cref="DataProtectionTokenProviderOptions"/> for this instance.
        /// </summary>
        /// <value>
        /// The <see cref="DataProtectionTokenProviderOptions"/> for this instance.
        /// </value>
        protected DataProtectionTokenProviderOptions Options { get; private set; } property
    }
}
```

```

public virtual async Task<bool> ValidateAsync(string purpose, string token, UserManager<TUser> manager, TUser user)
{
    try
    {
        var unprotectedData = Protector.Unprotect(Convert.FromBase64String(token));
        var ms = new MemoryStream(unprotectedData);
        using (var reader = ms.CreateReader())
        {
            var creationTime = reader.ReadDateTimeOffset();
            var expirationTime = creationTime + Options.TokenLifespan;
            if (expirationTime < DateTimeOffset.UtcNow)
            {
                return false;
            }
        }
    }
}

```

The default token life span is 1 day. You can see this yourself if you look at the source code of `DataProtectionTokenProviderOptions` class at the following link.

<https://github.com/aspnet/Identity/blob/release/2.2/src/Identity/DataProtectionTokenProviderOptions.cs>

```

namespace Microsoft.AspNetCore.Identity
{
    /// <summary>
    /// Contains options for the <see cref="DataProtectorTokenProvider{TUser}" />.
    /// </summary>
    public class DataProtectionTokenProviderOptions
    {
        /// <summary>
        /// Gets or sets the name of the <see cref="DataProtectorTokenProvider{TUser}" />. Default
        /// </summary>
        /// <value>
        /// The name of the <see cref="DataProtectorTokenProvider{TUser}" />.
        /// </value>
        public string Name { get; set; } = "DataProtectorTokenProvider";

        /// <summary>
        /// Gets or sets the amount of time a generated token remains valid. Defaults to 1 day.
        /// </summary>
        /// <value>
        /// The amount of time a generated token remains valid.
        /// </value>
        public TimeSpan TokenLifespan { get; set; } = TimeSpan.FromDays(1);
    }
}

```

```

services.Configure<DataProtectionTokenProviderOptions>(o =>
    o.TokenLifespan = TimeSpan.FromHours(5));

services.AddMvc(options =>

```

```

services.Configure<DataProtectionTokenProviderOptions>(o =>
    o.TokenLifespan = TimeSpan.FromHours(5));

```

Това означава, че всички токени имат продължителност 5 часа живот оттук нататък, не само за пароли, но и за имейли. Принципно токени на имейлите имат по-продължителна годност от токени на паролите.

Password Reset Token Lifetime

Changes token lifespan of all token types to 5 Hours

```
public void ConfigureServices(IServiceCollection services)
{
    // rest of the code

    services.Configure<DataProtectionTokenProviderOptions>(o =>
        o.TokenLifespan = TimeSpan.FromHours(5));

    // rest of the code
}
```

To change lifetime of just one type of token, we have to create a custom

- **DataProtectorTokenProvider**
- **DataProtectionTokenProviderOptions**

Може да персонализираме само токени на имейлите да имат продължителност 3 дена, което ще дискутираме в следващото видео.